



MASTER IN PHYSICAL SCIENCES
École Normale Supérieure de Lyon
Université Claude Bernard Lyon I

2026 Internship
DEUMIER Constantin
M1 Physics

Hebbian learning in bacterial reaction networks

Abstract : We model and simulate *in silico* a bacterial neural network in which computation emerges from purely local biochemical rules. Each cell communicates through diffusible spikes and adapts its response function through a Hebbian plasticity, realising a physical form of Kolmogorov-Arnold network. Stochastic simulations reproduce Pavlovian conditioning and several two-input logic gates, including XOR, made separable by a bias node. We finally discuss the limits of the approach, in particular the trade-off between heterogeneous connectivity and a uniform learning circuit.

keywords : *networks, biological neurons, bioinformatics, DNA*

Internship supervised by:

Angel Goni-Moreno

Lewis Grozinger

CNB: Centro Nacional de Biotecnología

July 10, 2026

Contents

1	Introduction	1
1.1	DNA reaction networks	2
1.1.1	Transcription and translation	2
1.1.2	Modelling transcription with the Hill function	3
1.2	Hebbian learning	4
1.3	Kolmogorov-Arnold networks (KANs)	5
2	The neural model	7
2.1	Stochastic simulation	7
2.2	The genetic circuit	7
2.3	Spike model	9
2.4	Hebbian dynamics	11
2.4.1	Link with KANs	11
2.4.2	Non-monotonicity of the activation function	13
2.5	Diffusion	13
3	Methodology	15
3.1	Network topology	15
3.2	Training and test phases	16
4	Results	16
4.1	Stimulus-response conditioning	16
4.2	Linearly separable logic gates	17
4.3	Non-linearly separable logic gates	19
5	Conclusion	20
A	Parameters	24

1 Introduction

Biological computing, or biocomputing, studies the ability of living systems to process, transmit and store information. Within this interdisciplinary framework, blending synthetic biology and mathematical modelling, the “Bio-Kan” project aims to design and simulate an *in silico* biological neural network. The goal of this internship is to study how complex logical operations can emerge from a consortium of genetically modified bacteria [1], relying solely on local biochemical rules. The system studied departs from standard artificial neural network architectures by modelling a liquid, stochastic system constrained by physical laws. This computing paradigm rests on four fundamental principles.

First, communication proceeds through temporal events, or spikes. The information flowing between cells is not encoded by molecular concentrations at equilibrium, but by the frequency and density of discrete events, expressed as sudden bursts of gene expression. This approach is akin to Random Pulse Computing [2] and offers two advantages: it exploits the intrinsically noisy nature of biochemical systems, and it is more energy-efficient, the cell spending resources only during brief spikes rather than by maintaining a continuous expression. This pulse-based communication places the model within the family of spiking neural networks [3], sometimes referred to as the third generation of neural networks, where information is no longer carried by a continuous activation but by the timing and frequency of spikes. This temporal coding, closer to the way biological neurons operate, is also the one exploited by STDP-type learning rules, which we will encounter later.

Second, the model relies on a spatial “reservoir”-type architecture [4, 5]. The network has no fixed wiring topology: the nodes, represented by the bacteria, communicate exclusively through the chemical diffusion of signalling molecules in the culture medium [6]. The strength of the interaction between two cells is therefore not an abstract parameter, but a variable set dynamically and probabilistically by their physical distance. Distributing the computation over several bacteria, rather than concentrating everything in a single cell, has a twofold benefit. On the one hand, it avoids molecular parallelism: instead of requiring many distinct, orthogonal molecular species within a single cell, each bacterium reuses the same small set of signals, the complexity arising from the spatial arrangement. On the other hand, it lets an informational complexity emerge from the collective.

Third, the architecture is distinguished by a nodal plasticity conceptually close to Kolmogorov-Arnold networks (KANs). Unlike classical deep-learning models, where adaptation proceeds by adjusting synaptic weights between fixed nodes, the intercellular connections are here imposed by the laws of molecular diffusion. Learning is consequently internalised: it is the activation function of the bacterium itself, determined by the state of its genetic circuits, that changes plastically in response to its stimulation history.

Finally, this system entails a dynamic and strictly local form of learning. Such a rule is first of all the natural form of learning in the living world. In the absence of any global optimisation algorithm such as gradient backpropagation [7], the network self-organises. This choice also has a scaling advantage: backpropagation requires a global, coordinated propagation of the gradient, whose cost grows with the number of cells, whereas a local rule parallelises naturally, each bacterium learning independently of the size of the network. The cells embed genetic circuits implementing mechanisms of memory, sensitisation and habituation based on Hebbian-inspired rules. They thus process information autonomously, without direct external supervision of their internal states.

This report presents the mathematical modelling of this system at the scale of stochastic reaction networks, using the Gillespie algorithm. It details the internal dynamics of a single bacterium and the diffusion coupling mechanisms, and finally demonstrates how a spatial network of five to ten bacteria, depending on the topology, can be conditioned to reproduce both linearly and non-linearly separable

logical operations, such as the XOR gate.

1.1 DNA reaction networks

Deoxyribonucleic acid (DNA) is the molecule that carries the genetic information of nearly all living organisms. Fundamentally, DNA acts as a storage polymer whose information is encoded by a sequence of four nucleotides (adenine, thymine, cytosine, guanine). Yet, beyond its passive role as long-term memory, DNA, together with the cellular machinery that reads and interprets it, constitutes a genuine information-processing system. In this view, the interactions between DNA, RNA and various proteins can be conceptualised as a network of programmable chemical reactions.

The idea of using DNA as a computing substrate is not new. In 1994, Leonard Adleman carried out the first experimental demonstration of DNA computing by solving a Hamiltonian path problem solely through the hybridisation of DNA strands *in vitro* [8]. Subsequently, the emergence of synthetic biology in the early 2000s made it possible to transpose these concepts *in vivo*. Pioneering works demonstrated that artificial genetic circuits could be built within bacteria, such as the bistable genetic switch (the toggle switch of Gardner *et al.*) [9] or the synthetic oscillator (the repressilator of Elowitz and Leibler) [10]. These advances proved that logic gates and rudimentary memories could be implemented by suitably arranging genes and their regulators [11].

To design and simulate bacterial neural networks capable of complex operations, these molecular dynamics must be modelled precisely. Understanding these reaction networks rests first on describing the processes that read genetic information, then on translating them into systems of mathematical equations.

1.1.1 Transcription and translation

A cell's functioning relies on the expression of its genes, that is, the conversion of the information encoded in DNA into functional molecules, the proteins. This flow of information, summarised by the "central dogma" of molecular biology, proceeds in two successive steps.

The first, transcription, is the synthesis of a messenger RNA (mRNA) molecule from a gene. An enzyme, the RNA polymerase, binds to a regulatory region upstream of the gene, the promoter, then runs along the DNA sequence to produce a complementary copy in the form of RNA. The second step, translation, sees this mRNA decoded by a ribosome that assembles, amino acid by amino acid, the corresponding protein.

From the standpoint of information processing, the decisive step is the initiation of transcription. The recruitment of the RNA polymerase onto the promoter can indeed be modulated by other proteins, the transcription factors, which bind to specific sites on the DNA: an activator promotes this recruitment and increases the production rate, whereas a repressor hinders it and lowers it. It is precisely this regulation, whereby the concentration of one protein controls the production rate of another, that allows genes to be wired into circuits and gives the system its computing capability. This capability is, moreover, not anecdotal: a network of chemical reactions has been shown to be a Turing-complete computing substrate, capable in principle of simulating any algorithm. This result, established for deterministic chemical kinetics by Magnasco (1997) [12], was later extended to stochastic reaction networks, with an arbitrarily small error probability, by Soloveichik *et al.* (2008) [13]. For modelling purposes, the details of translation are most often lumped together with those of transcription: one directly describes the production rate of a protein as a function of the concentration of its regulators.

1.1.2 Modelling transcription with the Hill function

The quantity of interest is therefore the synthesis rate of a protein as a function of the concentration of the regulator that controls its expression. This relationship is not linear but sigmoidal: it shows a threshold below which the gene remains practically off, an abrupt transition zone, then a saturation. The Hill function [14] is the mathematical expression that captures this behaviour.

Its origin is the assumption of cooperative binding: n regulator molecules are assumed to bind jointly to the promoter to change its state. Treating this binding at thermodynamic equilibrium, the fraction of occupied promoters can be expressed as a function of the regulator concentration x . For an activator, the production rate takes the form

$$f(x) = \beta \frac{x^n}{K^n + x^n}, \quad (1)$$

and for a repressor, the complementary form

$$f(x) = \beta \frac{K^n}{K^n + x^n}. \quad (2)$$

Three parameters suffice to characterise it. The prefactor β sets the maximal production rate (the saturation). The constant K is the half-activation threshold: the regulator concentration at which production reaches half of its maximal value. Finally, the Hill coefficient n controls the steepness of the transition: the larger n , the more abrupt the switch between the “off” and “on” states (figure 1).

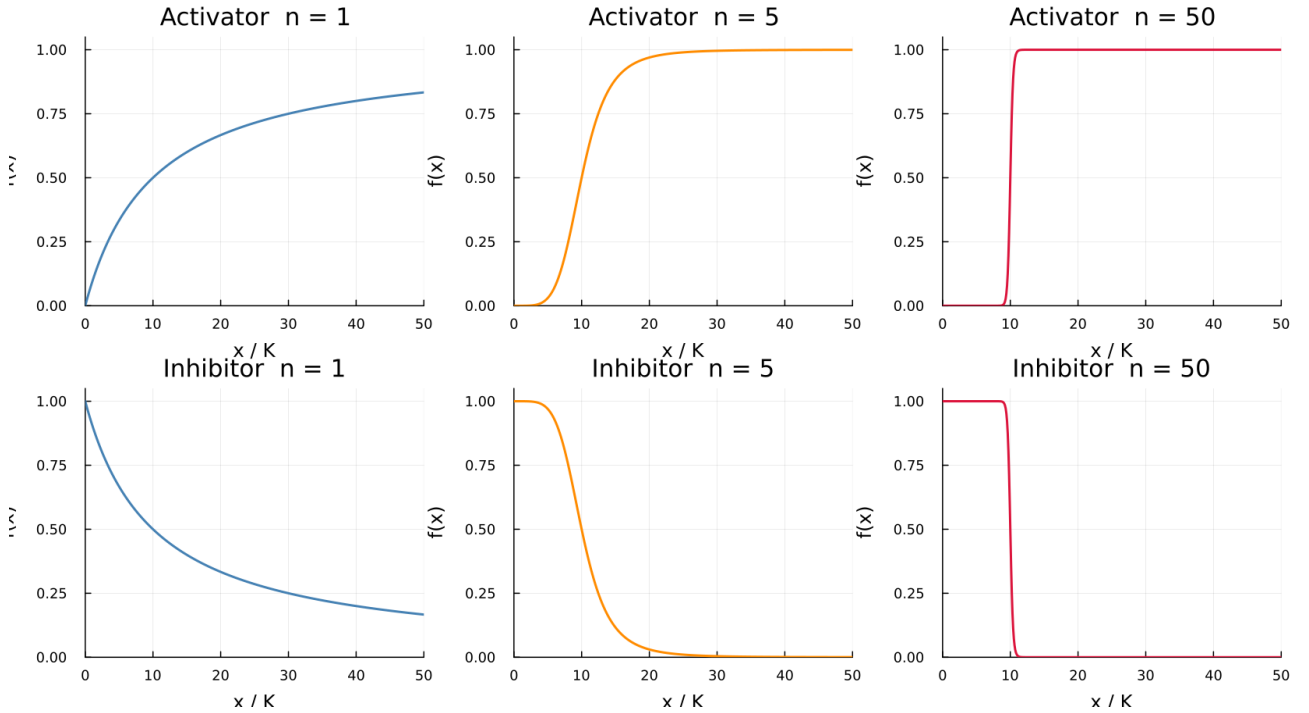


Figure 1: Activating Hill function $x^n/(K^n + x^n)$ (top) and repressing Hill function $K^n/(K^n + x^n)$ (bottom), for several values of the Hill coefficient n . The larger n , the steeper the transition near the threshold K , tending towards a step function.

As $n \rightarrow \infty$, the Hill function tends towards a step function and the gene behaves like an ideal switch, a natural support for a logic gate. It is this adjustable threshold nonlinearity, tuned continuously through n and K , that plays the role of the biological “activation function” of the network, the analogue of the

sigmoid of artificial neurons.

The protein concentration is, however, not set by this production rate alone: it results from the balance between production and degradation (dilution by cell growth, proteolysis). Writing γ for the degradation rate, the dynamics of the concentration P reads

$$\frac{dP}{dt} = f(x) - \gamma P, \quad (3)$$

whose steady state $P_{\text{eq}} = f(x)/\gamma$ reproduces, up to a scale factor, the Hill sigmoid.

Beyond a single gene, the richness of the mechanism comes from combining several Hill functions to build logic gates on concentrations [15]. Consider two regulators x and y , each summarised by its Hill function, close to 0 when absent and to 1 when present. Their product $H(x)H(y)$ is close to 1 only when both are present: this realises a logical AND. Their sum $H(x) + H(y)$ is high as soon as at least one is present: this realises a logical OR. Finally, a repression $1 - H(x)$ inverts an input and realises a NOT; in this report, the notation $1 - H$ merely denotes a repression and is not to be taken as a literal arithmetic operation. Since AND, OR and NOT form a complete set, any gate can be assembled by composition. For large n , each Hill function behaves like a binary threshold, so these combinations act as genuine logic gates on the concentrations. This is the principle exploited in the bacterial circuit, where, for instance, coincidence-based learning results from the product of two Hill functions, that is, an AND between two signals.

In its basic form, the Hill function is monotonic. By introducing higher-order terms, however, one obtains non-monotonic responses. Thus, adding to the denominator a term in x^{2n} ,

$$g(x) = \frac{x^n}{K_1^n + x^n + x^{2n}/K_2^{2n}}, \quad (4)$$

makes the rate increase at low concentration then decrease at high concentration: this is a biphasic response, or inhibition by excess regulator. The model will later rely on such a form, not for the activation function itself, but for the internal term that encodes memory and governs learning. These two non-monotonicities, that of learning and that of the activation, are distinct and will be treated separately.

1.2 Hebbian learning

The idea that learning could result from a simple local rule is an old one. As early as the beginning of the twentieth century, Pavlov's work on conditioning showed that an organism durably associates two stimuli presented together, without however identifying the physical substrate. It was Donald Hebb who, in 1949, proposed a cellular mechanism for this association in *The Organization of Behavior* [16].

His postulate has remained famous in a condensed form: neurons that fire together wire together. More precisely, when a neuron repeatedly takes part in the activation of one of its neighbours, the efficacy of the link between the two is strengthened. The essential property, the one that makes the mechanism so attractive for computation, is its locality: the modification depends only on the joint activity of the two cells involved, without any global supervision signal.

In its simplest form, this rule is written as a correlation between presynaptic activity x_i and postsynaptic activity x_j , namely $\Delta w_{ij} = \eta x_i x_j$. The change is incremental: each coincidence produces only a small change, set by the learning rate η . Learning therefore does not happen in a single presentation, but through the repeated presentation of the same data, each input-output pair reinforcing the association a little more until it settles. As such, it describes only a strengthening: the weights can only

grow and end up saturating, so that the network loses any ability to discriminate between its inputs. A viable Hebbian rule must therefore balance this potentiation with a form of weakening, in other words a forgetting [17, 18]. It is this strengthening/forgetting pair that we will implement explicitly at the bacterial level.

This plasticity first found an experimental substrate with the discovery of long-term potentiation by Bliss and Lømo (1973) [19]. Later work showed that the very sign of the modification depends on the temporal order of the spikes, a mechanism known as spike-timing-dependent plasticity (STDP) [20, 21]. When the presynaptic neuron fires just before the postsynaptic one, the link is strengthened; in the reverse order, it is weakened, the effect decreasing rapidly with the temporal gap. The synapse then behaves as a detector of coincidence and causality: it favours predictive connections, those where the activity of one cell genuinely contributes to triggering that of its neighbour. It is this temporal reading, based on the coincidence of discrete events, that directly inspires the reinforcement rule of our network and makes it possible to build the oriented associations needed for non-trivial operations such as the XOR gate.

1.3 Kolmogorov-Arnold networks (KANs)

Kolmogorov-Arnold networks (KANs) [22] rest on a result from approximation theory, the Kolmogorov-Arnold representation theorem [23, 24], which states that any continuous function of several variables can be written as a composition of continuous functions of a single variable and of additions. Formally, for a function f of n variables,

$$f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \varphi_{q,p}(x_p) \right), \quad (5)$$

where the $\varphi_{q,p}$ and the Φ_q are functions of a single variable. The only truly non-trivial ingredient of the computation is therefore a set of single-variable functions, the interactions between variables reducing to simple sums.

This structure inverts the role of the components compared with a classical multilayer perceptron. In the latter, each connection carries an adjustable scalar weight and each neuron applies a fixed activation function (sigmoid, ReLU) to the weighted sum of its inputs; learning amounts to tuning these weights. In a KAN, it is on the contrary the activation functions themselves that are learned and placed on the connections, most often parametrised by splines, while the nodes merely add. The computation is then no longer carried by a weight matrix but by a set of single-variable functions, individually more expressive (figure 2).

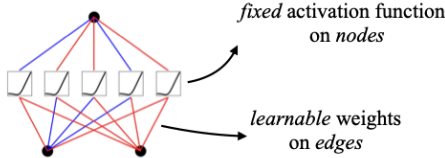
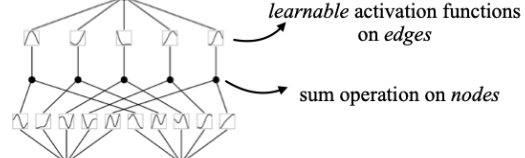
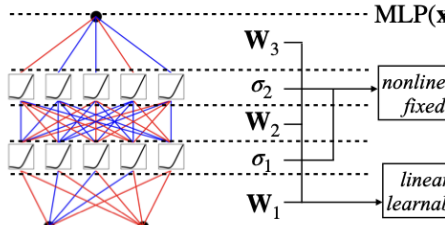
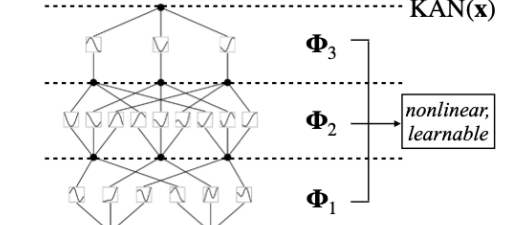
Model	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Theorem	Universal Approximation Theorem	Kolmogorov-Arnold Representation Theorem
Formula (Shallow)	$f(\mathbf{x}) \approx \sum_{i=1}^{N(e)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Model (Shallow)	(a)  fixed activation functions on nodes learnable weights on edges	(b)  learnable activation functions on edges sum operation on nodes
Formula (Deep)	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Model (Deep)	(c)  MLP(x) $\mathbf{W}_3$ σ_2 nonlinear, fixed $\mathbf{W}_2$ σ_1 $\mathbf{W}_1$ linear, learnable $\mathbf{x}$	(d)  KAN(x) Φ_3 Φ_2 nonlinear, learnable Φ_1 $\mathbf{x}$

Figure 2: Comparison between a multilayer perceptron (MLP, left) and a Kolmogorov-Arnold network (KAN, right), in shallow (top) and deep (bottom) versions. The MLP, justified by the universal approximation theorem, places fixed activation functions on the nodes and learns linear weights on the edges; the KAN, grounded in the Kolmogorov-Arnold representation theorem, places learned activation functions on the edges and merely sums at the nodes. Figure taken from Liu *et al.* (2024) [22].

This particular distribution of learning is precisely what makes the KAN formalism relevant for a bacterial consortium. In an artificial neural network, flexibility comes from the possibility of strengthening one weight relative to the others, that is, of independently modulating the gain of each connection. But a bacterium has no such mechanism: the coupling strength between two cells is set by molecular diffusion and their physical distance, and is not a parameter the cell could freely adjust. What a bacterium can change, on the other hand, through the plasticity of its genetic circuits, is its own response function, the way it transforms the signals it receives into an emitted signal.

Learning thus naturally lodges where it is biologically feasible: not in non-existent connection weights, but in the activation functions of the nodes, exactly as in a KAN. The bacterial network thereby realises a physical form of Kolmogorov-Arnold network, where each bacterium plays the role of a learned function and diffusion that of the summation and of the fixed wiring.

It remains to define how to concretely combine this KAN-type plasticity with the Hebbian learning rule; this is precisely the subject of the next part, devoted to the neural model itself.

2 The neural model

2.1 Stochastic simulation

At the scale of a single cell, the chemical species involved in gene expression (messenger RNAs, proteins, signalling molecules) are present in very small numbers, sometimes only a few dozen copies. In this regime, describing their evolution with continuous concentrations and deterministic differential equations, the so-called mean-field approach, no longer makes sense: the reactions are discrete and random events, and the resulting fluctuations are not a negligible noise but an essential component of the dynamics. This is all the more true here since the computing paradigm relies entirely on this noisy character, the information being carried by discrete spikes. A stochastic description is therefore required.

The exact object describing such a system is the chemical master equation, which follows the evolution of the probability of each state of the system, that is, the vector of the molecule numbers of each species. This equation is generally impossible to solve analytically. Rather than seeking the full distribution, the Gillespie algorithm, or SSA for Stochastic Simulation Algorithm [25], samples individual, statistically exact trajectories from it.

Its principle is simple. To each possible reaction j is associated a propensity a_j , which measures the probability per unit time that it occurs in the current state; for mass-action kinetics, this propensity is proportional to the number of available combinations of reactant molecules. The algorithm then proceeds by two random draws at each step. The first determines when the next reaction takes place: the waiting time follows an exponential law with parameter $a_0 = \sum_j a_j$. The second determines which one occurs, each reaction being chosen with probability a_j/a_0 . The molecule numbers are updated accordingly, time is advanced, and the process repeats. One thus obtains a sequence of discrete events whose statistics exactly reproduce those of the master equation, and which naturally restores the granular, noisy character of gene expression.

In practice, the simulation proceeds by fixed time steps. At each step, the internal chemistry of every bacterium is first advanced by the SSA over the interval, then a diffusion step redistributes among the cells the molecules emitted into the medium. Chemistry and diffusion are thus treated in alternation rather than within a single stochastic process, a separation justified by the gap between the fast internal kinetics and the slower intercellular transport.

The implementation relies on the Julia language and the Catalyst.jl library [26], from the SciML scientific-computing ecosystem. Catalyst makes it possible to define a reaction network symbolically, by writing the chemical reactions directly, then automatically generates the propensities and the corresponding simulation code, executed by an SSA solver. The choice of Julia is motivated by performance: it produces fast compiled code, a decisive point since studying the network requires simulating a large number of trajectories, both to accumulate statistics and to sweep the parameter space.

2.2 The genetic circuit

The behaviour of a bacterium is described by a single reaction network, identical for all cells, whose species are listed in Table 1 and reactions in Table 2. To lighten the notation, we write the activating Hill function introduced above $H_p(x, K) = x^p/(K^p + x^p)$, and we denote the associated repression $1 - H_p$. We further adopt the following conventions, detailed in the appendix: v denotes a maximal transcription rate, α an emission upon a spike, β a translation, δ an internal degradation, γ an elimination in the medium, and n, m, l Hill coefficients.

Symbol	Type	Role
M	internal	activation variable (triggers the spike)
S	internal	memory, learned state
E_{int}	internal	echo of the cell's own activity
$\text{mRNA}_M, \text{mRNA}_S$	internal	messenger RNAs of M and S
D / D_{diff}	diffusible	excitatory input
$E_{\text{ext}} / E_{\text{ext,diff}}$	diffusible	external echo (learning)
C / C_{diff}	diffusible	forgetting signal
O / O_{diff}	diffusible	inhibitory output (strong spike)

Table 1: Species of the circuit. The diffusible species exist in an internal (perceived) form and in a diff form (emitted into the medium).

Reaction	Rate	Role
<i>Regulated transcription</i>		
$\emptyset \rightarrow \text{mRNA}_M$	ρ_M	integration of input + memory, repressed by O coincidence-based learning $E_{\text{int}} \times E_{\text{ext}}$ forgetting driven by C
$\emptyset \rightarrow \text{mRNA}_S$	ρ_S	
$S \rightarrow \emptyset$	r_{forget}	
<i>Translation</i>		
$\text{mRNA}_M \rightarrow \text{mRNA}_M + M$	β_M	translation of M
$\text{mRNA}_S \rightarrow \text{mRNA}_S + S$	β_S	translation of S
<i>Internal degradations</i>		
$X \rightarrow \emptyset$	δ_X	$X \in \{\text{mRNA}_M, \text{mRNA}_S, M, E_{\text{int}}, E_{\text{ext}}, C, S, D, O\}$
<i>Spikes: emission</i> ($\sigma_w = H_n(M, K_{\text{spike}})$, $\sigma_s = H_n(M, K_{\text{spike}}^O)$)		
$\emptyset \rightarrow D_{\text{diff}}$	$\alpha_D \sigma_w$	emission of the input
$\emptyset \rightarrow E_{\text{ext,diff}}$	$\alpha_E \sigma_w$	emission of the external echo
$\emptyset \rightarrow C_{\text{diff}}$	$\alpha_C \sigma_w$	emission of the forgetting signal
$\emptyset \rightarrow E_{\text{int}}$	$\alpha_{E_{\text{int}}} \sigma_w$	internal echo
$\emptyset \rightarrow O_{\text{diff}}$	$\alpha_O \sigma_s$	emission of the output (strong spike)
<i>Elimination in the medium</i>		
$D_{\text{diff}} \rightarrow \emptyset$	$\gamma_D H_n(D_{\text{diff}}, K_D^{\text{diff}})$	saturating
$E_{\text{ext,diff}} \rightarrow \emptyset$	γ_E	linear
$C_{\text{diff}} \rightarrow \emptyset$	γ_C	linear
$O_{\text{diff}} \rightarrow \emptyset$	γ_O	linear

Table 2: Reactions of the genetic circuit. The regulated rates ρ_M , ρ_S and r_{forget} are made explicit below.

The three regulated rates read

$$\rho_M = [v_M H_l(D, K_D) + v_L G(S)] (1 - H_n(O, K_O)) + v_0, \quad G(S) = \frac{S^m}{K_a^m + S^m + S^{2m}/K_i^{2m}}, \quad (6)$$

$$\rho_S = v_S H_n(E_{\text{int}}, K_{E_{\text{int}}}) H_n(E_{\text{ext}}, K_{E_{\text{ext}}}), \quad r_{\text{forget}} = v_C H_n(C, K_C) H_4(S, S_0). \quad (7)$$

The distinction between internal and diffusible species is at the heart of the model. Each of the four diffusible signals exists in two forms: a form emitted into the medium, written with the subscript diff, released upon a spike, and an internal form that the cell perceives. The passage from one to the other

is not instantaneous: it is governed by diffusion in the culture medium, which depends on the distance between cells and is the subject of the section devoted to diffusion. A cell therefore emits the diff form, which diffuses, and neighbouring cells capture a fraction of it in the internal form. It is this mechanism, and not a predefined wiring, that realises the connections of the network. Finally, note that the learning term $G(S)$ is a non-monotonic function of the memory S , increasing then decreasing (figure 3): the sensitisation provided by the memory passes through a maximum before declining. This choice is a stability guard: the decrease at high S prevents the memory from diverging. The peak then acts as a ceiling, the intended operation remaining on the rising flank, where strengthening the memory does increase the cell's responsiveness.

2.3 Spike model

The core of each bacterium is the protein M , produced continuously at rate ρ_M and degraded at a constant rate. Its equilibrium level depends only on the received input D , the memory S and the basal rate v_{ground} . Depending on the combination of these quantities, M settles at distinct levels (figure 3): silent in the absence of input and memory, held at an intermediate level by the memory S or the basal term, or strongly activated under the effect of an input. This level of M acts as the activation variable of the cell, and it is its crossing of a threshold that triggers a spike.

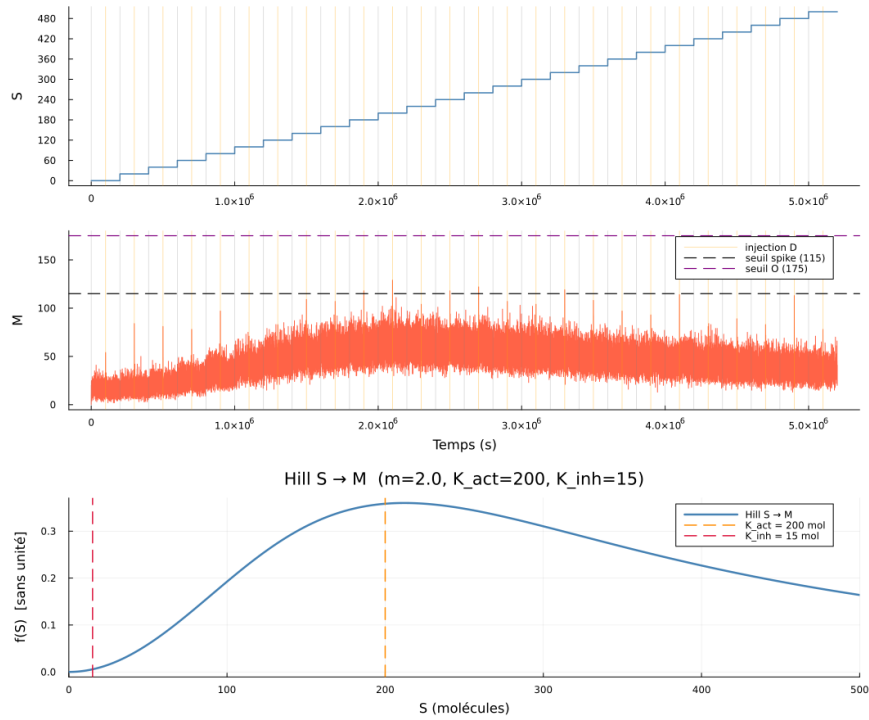


Figure 3: From top to bottom: the memory S imposed in increasing steps; the stochastic trace of M in response to an injection of D at each step, with the spike and output thresholds; and the non-monotonic Hill function $G(S)$ that weights the effect of the memory. The level of M increases then decreases with S : this is the non-monotonicity of learning, not to be confused with that of the activation function.

Rather than implementing a genuine chemical excitable circuit in each bacterium, we realise the spike in software: a threshold on M is tested at each simulation step, and when it is crossed the cell emits its signals into the medium for a fixed duration, before entering a refractory period that prevents it from

spiking again immediately. This choice is above all pragmatic: an excitable circuit would add many species and fast reactions, weighing down the stochastic simulation, whereas the software threshold cleanly isolates the abstraction of the spiking neuron. The emission can be seen as a sigmoidal function $\sigma(M) = H_n(M, K_{\text{spike}})$ of the activation level, the software threshold being its sharp limit. To preserve the stochastic character of the impulses, the duration of each spike is drawn with Gaussian noise.

Two thresholds are at play (figure 4). The first, K_{spike} , triggers the ordinary spike: the cell then releases into the medium the excitatory input D_{diff} intended for its neighbours, the external echo $E_{\text{ext,diff}}$ that will serve coincidence-based learning, and the forgetting signal C_{diff} ; it simultaneously re-emits its internal echo E_{int} , which keeps the trace of its own activity. The second threshold, higher, K_{spike}^O , corresponds to a strong spike that additionally releases the inhibitory output signal O_{diff} . Finally, a delay is applied to the propagation of some of these signals, the input D_{diff} and the forgetting signal C_{diff} , before they reach the neighbouring cells; the others, the external echo $E_{\text{ext,diff}}$ and the output O_{diff} , propagate without delay so as to arrive in phase. The justification for this choice, related to the travel time of the molecules in the medium, is given in the section devoted to diffusion.

Circuit 1 : Local integration and spike emission

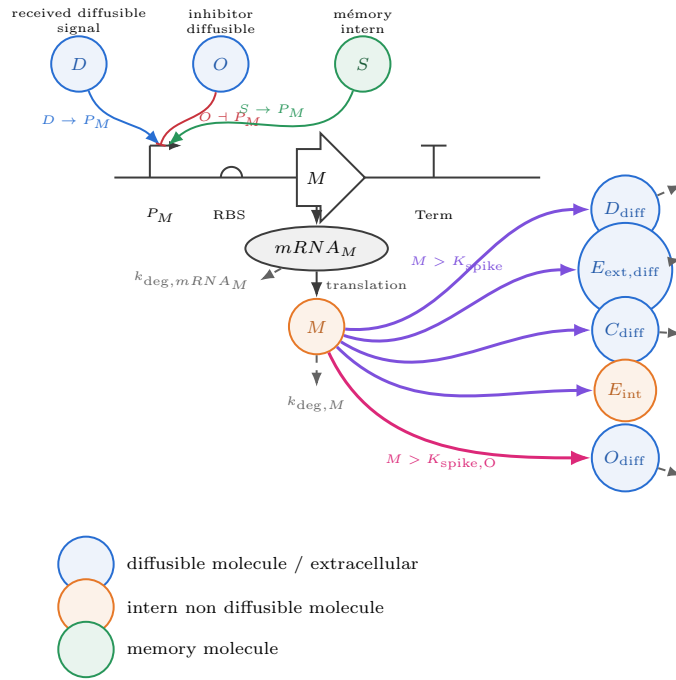


Figure 4: Genetic circuit of a bacterium. The promoter of M integrates the received input D , the memory S and the inhibitory signal O ; above a threshold on M , the cell emits the diffusible signals (D_{diff} , $E_{\text{ext,diff}}$, C_{diff} , internal echo E_{int}), and above a higher threshold the output signal O_{diff} .

This recourse to a software spike is a simplification, not a limitation of principle. A transient impulse can indeed be generated chemically by an incoherent feed-forward loop (IFFL) [27, 28]: an input signal activates both the output and a delayed repressor of that same output, so that the output rises briefly before being switched off, which produces a spike. The software spike could therefore, eventually, be replaced by an IFFL embedded in each cell, without changing the logic of the network.

2.4 Hebbian dynamics

2.4.1 Link with KANs

The learning rule is simply stated at the scale of a single cell. When a bacterium spikes and one of its neighbours spikes just after, it means that the signal of the first bacterium has been heard and has contributed to triggering this reaction: its memory S is strengthened. Conversely, each spike received without the cell having contributed to it tends to erase this memory. Learning therefore arises from the competition between a reinforcement by causal coincidence and a forgetting by mere exposure. The trace of the cell's own spike is the internal echo E_{int} , that of its neighbours the external echo E_{ext} , and the exposure the forgetting signal C . The genetic circuit realising this rule is shown in figure 5, and the temporal evolution of these signals in figure 6.

Circuit 2 : hebbien local learning

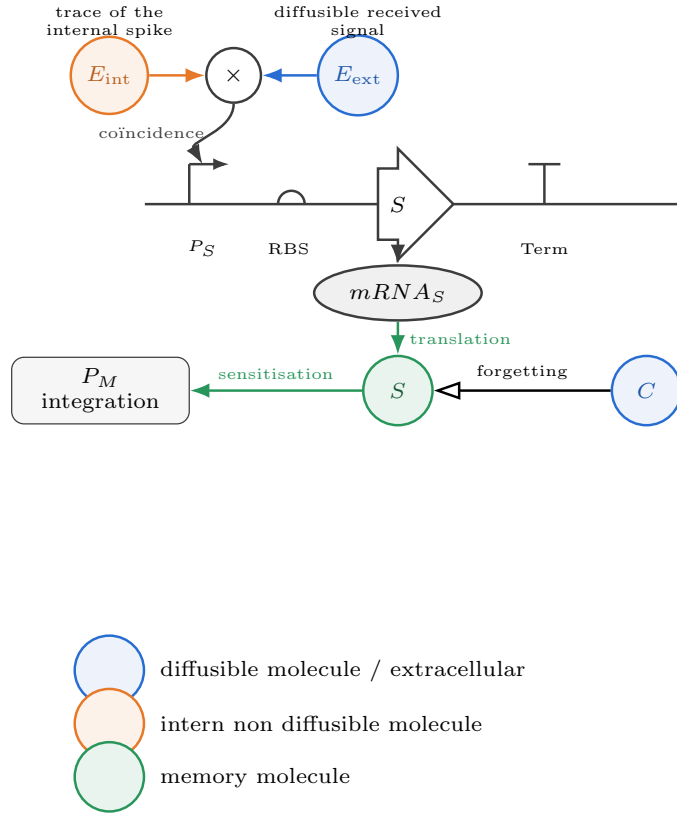


Figure 5: Genetic circuit of Hebbian learning (Circuit 2). The coincidence between the internal echo E_{int} and the external echo E_{ext} activates the production of the memory S ; the signal C degrades S (forgetting); in return, S modulates the promoter P_M of the integration circuit (sensitisation).

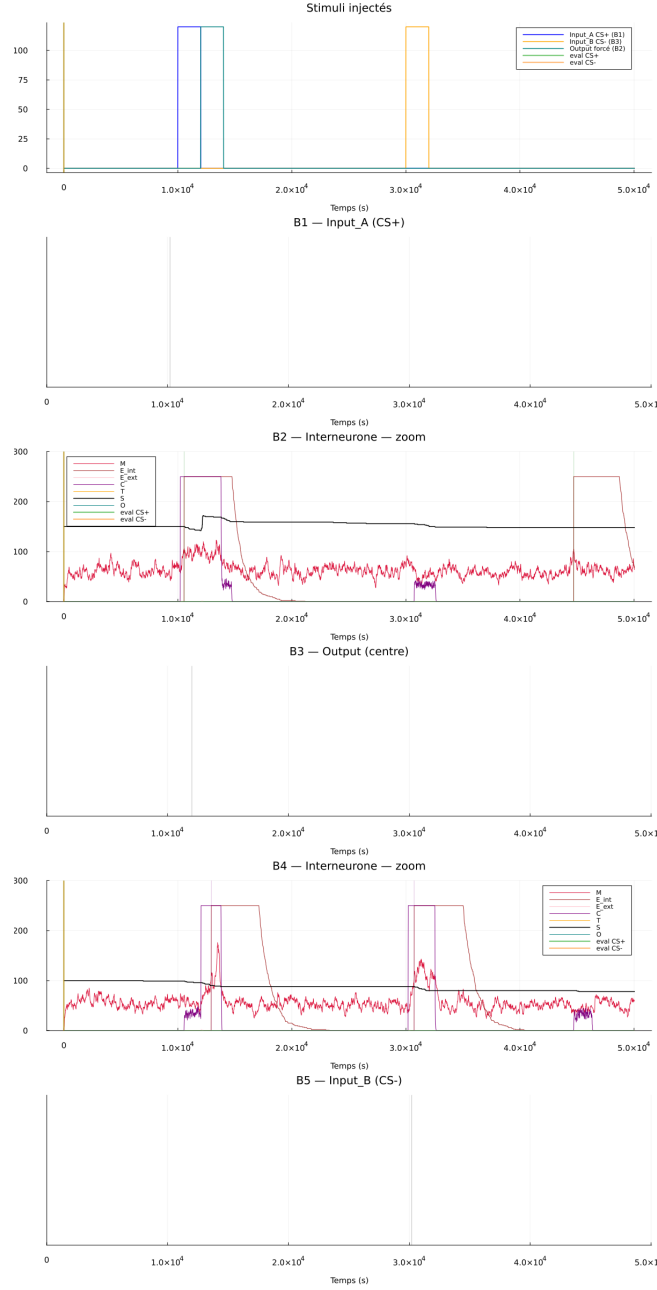


Figure 6: Conditioning on the line topology (5 bacteria), where only input A is associated with the output. At the top, the injected stimuli; below, the species of each bacterium (M in black, together with E_{int} , E_{ext} , C and S); only the interneurons B2 and B4 carry an internal dynamics. At each presentation of A, the output is forced to spike and its external echo E_{ext} diffuses towards both interneurons. Only the interneuron on the A side simultaneously receives an internal echo E_{int} , since only A has emitted a spike, propagated to it: the coincidence $E_{\text{int}} \times E_{\text{ext}}$ there strengthens the memory S , which grows. The interneuron on the B side, by contrast, does not spike; it does, however, receive the forgetting signal C emitted by the active interneuron and diffused far, so that its memory S decreases. The network thus distinguishes the pathway that predicts the output, reinforced, from the other, forgotten.

This rule is Hebbian in the strict sense. It is first of all local: it depends only on the activity of the cell and that of its immediate neighbours, without any global error signal. It then rests on temporal

coincidence, and even on its order: it is the fact that the cell's spike precedes that of its neighbour, and not the reverse, that triggers the reinforcement. One finds here exactly the principle of STDP, where a cause strengthens the link that leads to its effect. Forgetting by exposure, for its part, plays the role of the depression term, indispensable to prevent saturation, mentioned above.

There remains a difficulty. In a classical neural network, this reinforcement would adjust the weight of the connection between the cell and its neighbour. But a bacterium has no adjustable weight: its links are fixed by diffusion, and the learned quantity therefore cannot be stored in the connection. It is then transferred into the internal memory S of the cell, which reshapes its own response function. This shift of learning, from the connections to the nodal functions, is precisely what defines a Kolmogorov-Arnold network. The model is thus Hebbian by its local coincidence rule, and KAN by the place where this rule inscribes what it learns.

2.4.2 Non-monotonicity of the activation function

A monotonic activation function provides only a single decision threshold, and therefore separates only linearly separable classes. To go beyond this limit, the activation must be non-monotonic: active only within a window of its input, it offers two decision fronts instead of one.

There is no need for this to learn an arbitrary function, like the spline of a KAN: the simplest window, a rectangular crenel, already has these two fronts. A single crenel per node suffices, the expressivity then coming from the number of nodes rather than from the richness of each, a superposition of crenels approximating any continuous function.

In our model, this crenel is carried by the two spike thresholds. The first, K_{spike} , opens the window: the cell emits the excitatory signal D . The second, K_{spike}^O , higher, can only be crossed after the first and triggers the inhibitory signal O . Now O propagates without delay, whereas D is delayed: although produced after it, O reaches the neighbours before D and short-circuits the excitation. Above K_{spike}^O , this fast inhibition cancels the slow excitation and closes the window. The useful activity is thus confined to the band $K_{\text{spike}} < M < K_{\text{spike}}^O$, a crenel whose closing is set by the choice of the delays.

2.5 Diffusion

Communication between bacteria proceeds through the diffusion of molecules in the medium. The phenomenon involves two characteristic times: the time for the chemical integration of the genetic circuit and the travel time of a molecule between two cells. Modelling it faithfully raises a difficulty of both cost and regime.

Two classical descriptions are in principle available, but neither is suitable here. The first, the explicit Brownian tracking of each molecule, is exact but of prohibitive cost: one would have to simulate the random walks of a very large number of particles, at each step and for each cell. The second, Fick's law and its continuous reaction-diffusion equation, on the contrary assumes continuous concentrations, that is, the thermodynamic limit of a large number of molecules. But the system operates precisely outside this limit: the species are in very small numbers, and it is the discrete and noisy character of the signals that carries the information. A continuous description would erase exactly what the model seeks to exploit.

We therefore adopt an intermediate solution. Rather than simulating diffusion, we precompute, for each pair of cells, the probability that a molecule emitted by one reaches the other. This probability follows from the steady-state profile of diffusion with degradation,

$$p = \frac{R}{r} e^{-(r-R)/\lambda}, \quad \lambda = \sqrt{D/\gamma}, \quad (8)$$

where r is the distance between cells, R their radius and λ the diffusion length, that is, the typical distance travelled before degradation. Diffusion then reduces to a stochastic transfer: at each emission, the number of molecules received is drawn from a binomial law with parameter p . This choice preserves the essential physics, the screened $1/r$ profile and the range set by λ , as well as the discrete character of the signals, at a much reduced cost. It is hybrid, neither fully microscopic nor continuous, but suited to the regime of the model.

To this transfer is added a fixed delay, which represents the travel time of a molecule between two cells. It is applied only to the signals D_{diff} and C_{diff} ; the external echo $E_{\text{ext,diff}}$ and the output O_{diff} are on the contrary transmitted without delay, so as to arrive in phase with the activity they accompany. This single delay is a coarse approximation of the full distribution of first-passage times, of which it retains only the order of magnitude.

Finally, a constraint is imposed outside the chemistry: a diffusible molecule emitted by a cell can never be re-integrated by that same cell. This exclusion is enforced at the software level, the coupling of a cell with itself being set to zero. It has no physical justification, since nothing would prevent a bacterium from reabsorbing part of what it secretes. It is nonetheless necessary to avoid self-excitation, that is, to prevent a cell from indefinitely sustaining its own spike. It is therefore not an optimal choice, but a compromise indispensable to the proper functioning of the network.

The whole model, the two circuits carried by each cell and their coupling through the diffusion fields, is summarised in figure 7.

Two-bacterium Bio-KAN reservoir architecture

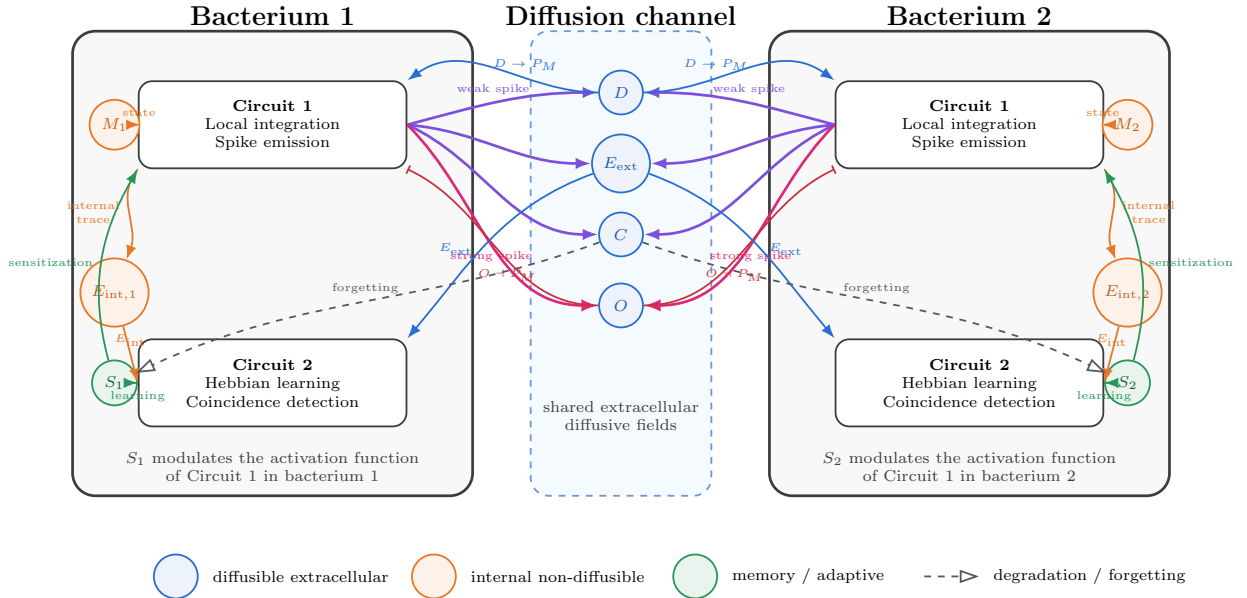


Figure 7: Complete architecture of the Bio-KAN reservoir for two coupled bacteria. Each cell carries Circuit 1 (integration and spike emission) and Circuit 2 (Hebbian learning); they communicate through the shared diffusible fields (D , E_{ext} , C , O). The memory S of each cell modulates the activation function of its own Circuit 1.

3 Methodology

3.1 Network topology

All the bacteria carry the same circuit; their function depends on their position in the network and on how they are driven. We distinguish the input nodes, whose spiking is imposed by the stimulus and which inject the information; the output node, whose activity is read as the result, forced during training and free at test; and the intermediate nodes, or interneurons, free, which constitute the learning layer of the network. A final type, the bias node, provides a constant reference: it takes the form either of a permanent static input or of a pacemaker that paces the network.

For the input and output nodes, the Hebbian circuit is of no importance: since their activity is imposed from outside, their internal learning dynamics does not come into play. Only the intermediate nodes genuinely exploit the Hebbian plasticity.

Three topologies of increasing complexity are studied (figure 8).

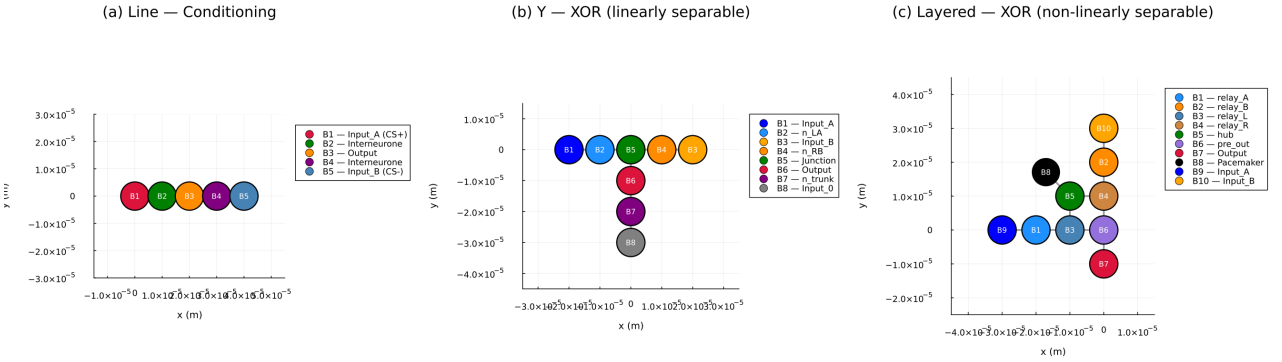


Figure 8: The three topologies studied: the five-node line (conditioning), the eight-node T with the bias input Input_0 (linearly separable XOR), and the ten-node layered network with a pacemaker (non-linearly separable XOR).

The first is a simple line of five bacteria: two inputs at the ends, an output node at the centre, and one interneuron on each side. It does not aim at a logic gate but at an elementary associative stimulus-response conditioning, and serves to validate the learning rule.

The second, in a T shape, adds a third input node, Input_0 , active only when both inputs are absent. This node makes XOR linearly separable. The four configurations of XOR cannot be separated by any hyperplane in the plane of the two inputs [29]; by assigning a coordinate of its own to the $(0,0)$ case, one embeds these points in a three-dimensional space where a single hyperplane separates them, the $(0,0)$ point being set apart along the new axis. This is a direct illustration of Cover’s theorem [30], according to which a classification problem projected non-linearly into a higher-dimensional space becomes more likely to be linearly separable. The nonlinearity is then offloaded into the embedding, and a linear reading of the output suffices.

The third, layered, replaces this static bias by an active pacemaker, which paces the network without providing a coordinate distinguishing the $(0,0)$ case. XOR therefore remains non-linearly separable there, and the network must solve it through the non-monotonicity of the activation function itself, the crenel described above. The last two topologies thus illustrate two opposite routes to the same gate: the linearising embedding on the one hand, the nodal non-monotonicity on the other.

3.2 Training and test phases

The experiment comprises two successive phases, separated by a short pause: a training phase, where the network learns, and a test phase, where what it has learned is evaluated. Both phases proceed in the same way. At each trial, one of the four input configurations (none, A only, B only, A and B) is presented; an epoch consists of the four configurations presented once each, in a random order, and the experiment repeats this over many epochs, each drawing a new random order.

Presenting an input consists in injecting the excitatory signal D into the corresponding node, which makes it spike and propagate D into the network.

During training, the expected output is also imposed when the truth table calls for a spike. This forcing is applied with a fixed delay Δ_{io} after the inputs, so as to install the causal order “input then output” on which coincidence-based reinforcement relies.

Unlike the other nodes, the spike of the output emits only the external echo E_{ext} , and not the excitatory signal D . This avoids it re-injecting excitation into the network, while still providing the echo needed for learning.

To this are added absence signals. An input node emits the inhibitory signal O when its stimulus is absent, in both phases; the output node emits O when the expected output is zero, but only during training. These signals teach the network when to stay silent. Throughout this phase, the memory S remains plastic.

The test phase begins after the pause and switches off plasticity: the memory S of each learning node is frozen at the mean of its values over the last training epochs, which denoises it and ensures that the measured behaviour reflects only what has been learned. The input and output nodes, whose memory has no effect in any case, are given an arbitrary value of S .

The output is then no longer forced: only the inputs and the pacemaker remain driven, while the output node is left free and its activity is read. The evaluation windows are lengthened, and a spike of the output is counted only if it is not cancelled by a concomitant inhibitory signal O .

4 Results

4.1 Stimulus-response conditioning

We apply the conditioning protocol [31, 32] to the line topology, in which only the two interneurons (B2 and B4) are plastic. The output B3 must learn to spike for the conditioned stimulus (CS+) and to stay silent for the other (CS−). Two symmetric experiments are run: in one, the CS+ is input A, in the other input B. After 40 training epochs, the network is evaluated over 50 test epochs, measuring the fraction of trials in which the output spikes.

The network clearly discriminates the two stimuli (figure 9). The output responds to the CS+ in 98% of trials (run A) and 96% (run B), against only 4% and 14% for the CS−. The rare errors are mostly false positives, where the output spills over onto a CS−. Discrimination is therefore successful, and symmetric: conditioning A or B gives comparable performance.

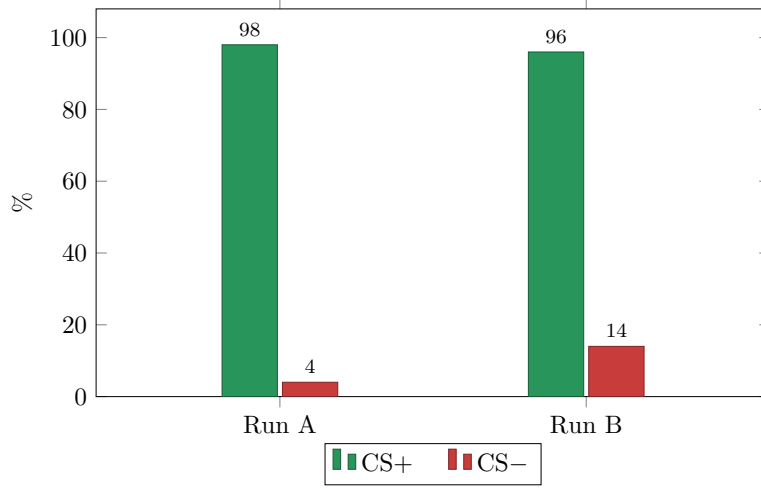


Figure 9: Output response rate (fraction of test trials in which it spikes) for the conditioned stimulus (CS+) and the neutral stimulus (CS-), in the two experiments (CS+ = A and CS+ = B). The strong response to CS+ and the weak response to CS- reflect successful discrimination.

Beyond performance, the point of interest is to see where the learning has been inscribed. The memory S of the interneurons at the end of training is strongly differentiated (Table 3): the interneuron on the side of the conditioned stimulus has a high memory, while the one on the other side is switched off. Conditioning A strengthens B2 and switches off B4; conditioning B does the reverse. It is therefore not the output that copies the input: it is a hidden interneuron that learns the association, on the right side, the other one forgetting it. This is exactly the mechanism of reinforcement by coincidence and forgetting by exposure presented above.

Learned memory S	B2 (A side)	B4 (B side)
CS+ = A	258	20
CS+ = B	20	226

Table 3: Mean memory S of the two interneurons at the end of training. The interneuron on the conditioned side is strengthened (high value), the other switched off.

4.2 Linearly separable logic gates

We then apply the T-shaped network to the sixteen two-input logic gates. This topology adds the bias node Input_0 which, as seen, makes XOR linearly separable. The score of a gate is the fraction of test trials in which the output responds correctly to the four input configurations.

Two sets of parameters of the Hebbian circuit lead to opposite behaviours (figure 10). The first performs well on the linearly separable gates (81% on average, up to 98% for NOR) but fails on the two non-linearly separable gates, XOR and XNOR, which remain at chance level (about 53%). The second unlocks XOR and XNOR (78% on average) but degrades the simple gates (62% on average). No setting succeeds on both families at once, it is a trade-off.

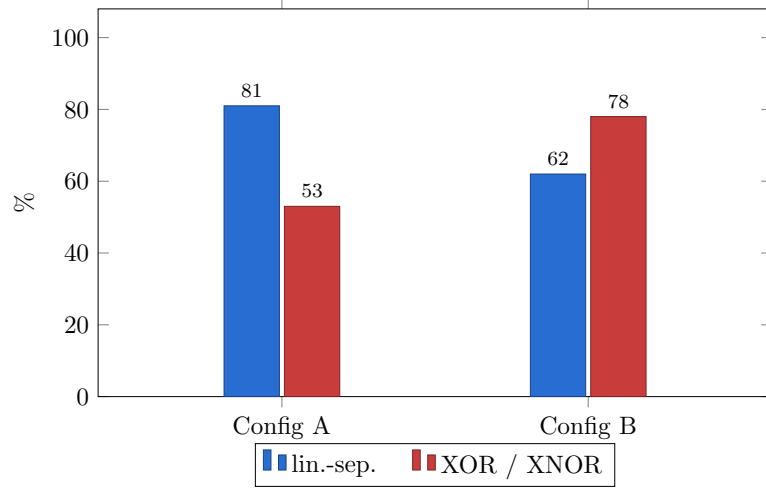


Figure 10: Mean gate score for each set of parameters, separating the fourteen linearly separable gates from the two non-linearly separable ones (XOR, XNOR). Configuration A succeeds on the former and fails on the latter; configuration B does the reverse. Neither succeeds on both.

The per-combination detail (Table 4) sharpens this contrast. In configuration A, almost all gates exceed 70%, except XOR (52%) and XNOR (55%) which stay at chance. In configuration B, XOR (85%) and XNOR (80%) are both solved, but several previously correct gates collapse, such as NOT_A_OR_B (40%) or A_OR_NOT_B (43%).

Gate	Config A					Config B				
	sc	A	B	AB	∅	sc	A	B	AB	∅
FALSE	95	100	98	94	86					–
TRUE	85	88	98	54	100	72	96	96	22	72
A	87	100	70	90	86	66	98	8	64	94
B	87	74	100	88	84	59	10	96	46	84
NOT_A	73	96	2	92	100	68	94	12	70	94
NOT_B	73	92	90	10	100	44	14	96	58	8
AND	64	40	30	98	86	75	62	58	90	90
NAND	65	76	50	34	100	61	90	92	56	4
OR	85	92	100	62	84	69	86	80	16	92
NOR	99	100	100	98	96	74	92	98	14	90
A_AND_NOT_B	71	92	92	10	88	69	78	96	10	90
NOT_A_AND_B	72	90	92	10	94	71	94	84	18	88
A_OR_NOT_B	88	98	72	88	94	43	90	4	64	12
NOT_A_OR_B	95	88	100	94	98	40	10	94	46	8
XOR	52	12	88	18	88	85	84	92	70	94
XNOR	55	8	30	80	100	80	70	65	85	100

Table 4: Score (%) of each gate and detail by input combination (A only, B only, A and B, none $\emptyset$), for the two sets of parameters. A value is shown in green if it exceeds 50, in red otherwise; the overall score is green only if all four combinations exceed 50, that is, if the gate is genuinely solved. FALSE was not re-run in config B.

Under this strict criterion, a gate is solved only if its four combinations all exceed 50%. The split then becomes clear: configuration A solves eight gates (FALSE, TRUE, A, B, OR, NOR, A_OR_NOT_B,

NOT_A_OR_B), all linearly separable, but neither XOR nor XNOR. Configuration B solves three, AND, XOR and XNOR, precisely the two non-linearly separable gates. Five gates (NOT_A, NOT_B, NAND, A_AND_NOT_B, NOT_A_AND_B) are solved by neither. Each configuration thus covers one family: config A the linearly separable gates, config B those that are not.

The origin of this trade-off is not learning itself, but a mismatch between the Hebbian circuit, identical for all cells, and the heterogeneous connectivity of the topology. In the T-shaped network, the nodes do not all have the same degree: the relays are of degree 2, but the Junction (B5), the only node that sees both input pathways, is of degree 3. Now what a cell receives, both reinforcement (E_{ext}) and forgetting (C), grows with its number of neighbours. A single set of parameters therefore cannot balance both the degree-2 nodes and the degree-3 node.

The learned memory confirms this directly (Table 5). In the regime that fails XOR, forgetting is calibrated for degree 2: the hidden nodes learn little (low S), and the degree-3 Junction, which absorbs the increased forgetting from its three neighbours, is over-erased and cannot learn the non-linear combination. In the regime that succeeds at XOR, forgetting is weakened so that this degree-3 node keeps its memory (high S); but the degree-2 nodes, now under-forgotten, over-learn, which degrades the simple gates.

Learned memory S	relays B2 / B4	Junction B5 (degree 3)	Input ₀ (B8)
Config A (fails)	17 / 40	67	48
Config B, XOR	190 / 185	147	149
Config B, XNOR	336 / 292	67	150

Table 5: Mean memory S of the hidden nodes. Config B strongly recruits the hidden layer and Input₀ (weakened forgetting), config A does not. The recruited node differs by gate: the Junction (degree 3) for XOR, the relays (degree 2) for XNOR.

This limitation suggests a natural fix: a degree-normalised plasticity, where reinforcement and forgetting would be scaled by the number of neighbours, so that a single circuit suits all connectivities. A topology with homogeneous degree would have the same effect.

4.3 Non-linearly separable logic gates

The third topology, layered with a pacemaker, removes the bias node Input₀. XOR is therefore no longer linearly separable there: the network can no longer rely on the Cover embedding and must solve the gate through the sole non-monotonicity of the activation function, the crenel introduced earlier. We apply the same protocol as in the previous section.

The scores are globally lower than on the T-shaped topology: most gates remain close to chance (Table 6). This difficult regime was expected, since without an embedding, and with a uniform Hebbian circuit on a deeper topology, learning has fewer levers. Two gates stand out: OR (64.5%) and above all XOR (60.5%).

Gate	sc	A	B	AB	$\emptyset$
FALSE	42.0	50	40	38	40
TRUE	61.5	62	62	74	48
A	60.5	66	58	58	60
B	48.5	32	56	60	46
NOT_A	52.5	32	66	50	62
NOT_B	54.5	74	32	46	66
AND	38.0	22	22	74	34
NAND	53.0	66	68	30	48
OR	64.5	60	62	86	50
NOR	39.0	28	32	42	54
A_AND_NOT_B	39.5	76	18	36	28
NOT_A_AND_B	48.0	40	66	44	42
A_OR_NOT_B	59.5	66	36	72	64
NOT_A_OR_B			—		
XOR	60.5	68	60	64	50
XNOR	45.5	32	24	58	68

Table 6: Score (%) and per-combination detail (A only, B only, A and B, none $\emptyset$) on the layered pacemaker topology, without a bias node. Same colour code as Table 4: a value is **green** if it exceeds 50, **red** otherwise; the overall score is **green** only if all four combinations exceed 50. The A, XOR and OR rows come from dedicated single-gate simulations; NOT_A_OR_B was not rerun.

The detail of XOR is instructive. The three configurations in which at least one input is present are handled correctly (A only 68%, B only 60%, A and B 64%), but the empty configuration (0, 0) remains exactly at chance level (50%). This is precisely the prediction of the argument of the previous section: the (0, 0) case is the one that the bias node Input_0 took charge of by assigning it a coordinate of its own. In its absence, the non-monotonicity of the activation separates the three active cases but leaves the empty case undecided.

The two routes to XOR are therefore not equivalent. The linearising embedding (previous section, XOR at 85%) remains markedly more effective than the nodal non-monotonicity (60%), which yields only a partial solution. The crenel demonstrates its principle, it pushes XOR above chance without any external bias, but does not on its own replace the contribution of an input dedicated to the (0, 0) case.

5 Conclusion

This work has proposed and simulated a bacterial neural network in which computation emerges from purely local rules. Each cell integrates its inputs through a genetic circuit, communicates by discrete spikes diffusing in the medium, and adapts its response through a Hebbian plasticity: reinforcement by causal coincidence, forgetting by exposure. The whole reads as a physical Kolmogorov-Arnold network, where learning lodges not in connection weights, which do not exist, but in the response function of each node.

The stochastic simulations first validate the mechanism on a conditioning task: the network durably associates a stimulus with a response, and the learned memory localises in the interneuron of the correct pathway, the other being switched off. Learning is indeed local, associative and selective.

Extended to two-input logic gates on a T-shaped topology, the network realises most functions, and the bias node Input_0 makes XOR and XNOR reachable, in accordance with Cover’s theorem. A more subtle result emerges, however: no single configuration solves the whole set of gates. This trade-off

is not a failure of learning, but the mismatch between a Hebbian circuit identical for all cells and a heterogeneous connectivity, a node receiving all the more reinforcement and forgetting as it has more neighbours. Each setting then balances only one class of degrees, separating, in this case, linearly and non-linearly separable gates into two regimes.

A second topology, layered and without a bias node, explored the other route to XOR, that of the non-monotonicity of the activation function. XOR there crosses the chance threshold without any embedding, which illustrates its principle, but across the whole set of gates only one is genuinely solved. These results therefore do not allow us to claim that this topology learns: they merely indicate that nodal non-monotonicity can initiate a separation, without rivalling the linearising embedding.

The identified trade-off points to the main perspective: a degree-normalised plasticity, or a topology with homogeneous degree, should reconcile the two families within a single network. Other directions extend the model: replacing the software spike by a genuine chemical excitable circuit (IFFL), exploiting the non-monotonicity of the activation function to solve XOR without embedding (pacemaker topology), and scaling up.

Beyond the logic gate, this work shows that a living substrate, noisy and coupled by diffusion, can learn and compute from local biochemical rules, a step towards programmable biological neural networks.

References

- [1] Alvin Tamsir, Jeffrey J. Tabor, and Christopher A. Voigt. “Robust Multicellular Computing Using Genetically Encoded NOR Gates and Chemical ‘Wires’”. In: *Nature* 469.7329 (2011), pp. 212–215.
- [2] Brian R. Gaines. “Stochastic Computing Systems”. In: *Advances in Information Systems Science*. Ed. by Julius T. Tou. Vol. 2. Springer, 1969, pp. 37–172.
- [3] Wolfgang Maass. “Networks of Spiking Neurons: The Third Generation of Neural Network Models”. In: *Neural Networks* 10.9 (1997), pp. 1659–1671.
- [4] Wolfgang Maass, Thomas Natschläger, and Henry Markram. “Real-Time Computing Without Stable States: A New Framework for Neural Computation Based on Perturbations”. In: *Neural Computation* 14.11 (2002), pp. 2531–2560.
- [5] Herbert Jaeger and Harald Haas. “Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication”. In: *Science* 304.5667 (2004), pp. 78–80.
- [6] Melissa B. Miller and Bonnie L. Bassler. “Quorum Sensing in Bacteria”. In: *Annual Review of Microbiology* 55 (2001), pp. 165–199.
- [7] Timothy P. Lillicrap et al. “Backpropagation and the Brain”. In: *Nature Reviews Neuroscience* 21.6 (2020), pp. 335–346.
- [8] Leonard M. Adleman. “Molecular Computation of Solutions to Combinatorial Problems”. In: *Science* 266.5187 (1994), pp. 1021–1024.
- [9] Timothy S. Gardner, Charles R. Cantor, and James J. Collins. “Construction of a Genetic Toggle Switch in *Escherichia coli*”. In: *Nature* 403.6767 (2000), pp. 339–342.
- [10] Michael B. Elowitz and Stanislas Leibler. “A Synthetic Oscillatory Network of Transcriptional Regulators”. In: *Nature* 403.6767 (2000), pp. 335–338.
- [11] Alec A. K. Nielsen et al. “Genetic Circuit Design Automation”. In: *Science* 352.6281 (2016), aac7341.

- [12] Marcelo O. Magnasco. “Chemical Kinetics is Turing Universal”. In: *Physical Review Letters* 78.6 (1997), pp. 1190–1193.
- [13] David Soloveichik et al. “Computation with Finite Stochastic Chemical Reaction Networks”. In: *Natural Computing* 7.4 (2008), pp. 615–633.
- [14] Archibald V. Hill. “The Possible Effects of the Aggregation of the Molecules of Haemoglobin on Its Dissociation Curves”. In: *Journal of Physiology* 40 (1910), pp. iv–vii.
- [15] Uri Alon. *An Introduction to Systems Biology: Design Principles of Biological Circuits*. Chapman and Hall/CRC, 2006.
- [16] Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.
- [17] Erkki Oja. “Simplified Neuron Model as a Principal Component Analyzer”. In: *Journal of Mathematical Biology* 15.3 (1982), pp. 267–273.
- [18] Elie L. Bienenstock, Leon N. Cooper, and Paul W. Munro. “Theory for the Development of Neuron Selectivity: Orientation Specificity and Binocular Interaction in Visual Cortex”. In: *Journal of Neuroscience* 2.1 (1982), pp. 32–48.
- [19] Timothy V. P. Bliss and Terje Lømo. “Long-Lasting Potentiation of Synaptic Transmission in the Dentate Area of the Anaesthetized Rabbit Following Stimulation of the Perforant Path”. In: *Journal of Physiology* 232.2 (1973), pp. 331–356.
- [20] Henry Markram et al. “Regulation of Synaptic Efficacy by Coincidence of Postsynaptic APs and EPSPs”. In: *Science* 275.5297 (1997), pp. 213–215.
- [21] Guo-qiang Bi and Mu-ming Poo. “Synaptic Modifications in Cultured Hippocampal Neurons: Dependence on Spike Timing, Synaptic Strength, and Postsynaptic Cell Type”. In: *Journal of Neuroscience* 18.24 (1998), pp. 10464–10472.
- [22] Ziming Liu et al. “KAN: Kolmogorov-Arnold Networks”. In: *arXiv preprint arXiv:2404.19756* (2024).
- [23] Andrey N. Kolmogorov. “On the Representation of Continuous Functions of Many Variables by Superposition of Continuous Functions of One Variable and Addition”. In: *Doklady Akademii Nauk SSSR* 114 (1957), pp. 953–956.
- [24] Vladimir I. Arnold. “On Functions of Three Variables”. In: *Doklady Akademii Nauk SSSR* 114 (1957), pp. 679–681.
- [25] Daniel T. Gillespie. “Exact Stochastic Simulation of Coupled Chemical Reactions”. In: *The Journal of Physical Chemistry* 81.25 (1977), pp. 2340–2361.
- [26] Torkel E. Loman et al. “Catalyst: Fast and Flexible Modeling of Reaction Networks”. In: *PLoS Computational Biology* 19.10 (2023), e1011530.
- [27] Shmoolik Mangan and Uri Alon. “Structure and Function of the Feed-Forward Loop Network Motif”. In: *Proceedings of the National Academy of Sciences* 100.21 (2003), pp. 11980–11985.
- [28] Uri Alon. “Network Motifs: Theory and Experimental Approaches”. In: *Nature Reviews Genetics* 8.6 (2007), pp. 450–461.
- [29] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.
- [30] Thomas M. Cover. “Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition”. In: *IEEE Transactions on Electronic Computers* EC-14.3 (1965), pp. 326–334.

- [31] Ilias Tagkopoulos, Yir-Chung Liu, and Saeed Tavazoie. “Predictive Behavior Within Microbial Genetic Networks”. In: *Science* 320.5881 (2008), pp. 1313–1317.
- [32] Amir Mitchell et al. “Adaptive Prediction of Environmental Changes by Microorganisms”. In: *Nature* 460.7252 (2009), pp. 220–224.

A Parameters

Table 7 summarises the parameters of the genetic circuit, with the correspondence between the report’s notation and the name used in the code, for both experiments: conditioning and logic gates (configuration B). Table 8 gathers the parameters of the software spike, of the diffusion and of the protocol. The few inactive species and parameters (including I) are omitted.

Notation	Code	Cond.	Gates	Pace.
<i>Hill exponents</i>				
n	<code>n</code>	15	15	15
m	<code>m</code>	1	2	2
l	<code>l</code>	1	1	1
<i>Maximal transcription rates</i>				
v_M	<code>v_max_M</code>	0.85	2.5	2.7
v_L	<code>v_max_learn</code>	0.95	0.95	0.9
v_0	<code>v_ground</code>	0.13	0.12	0.16
v_S	<code>v_max_ES</code>	1	0.7	0.7
v_C	<code>v_max_CS</code>	0.002	1.5×10^{-5}	0.018
<i>Hill thresholds</i>				
K_D	<code>K_DM</code>	10	74	74
K_a	<code>K_act_S</code>	200	200	200
K_i	<code>K_inh_S</code>	15	15	15
$K_{E_{\text{int}}}$	<code>K_EiS</code>	20	20	20
$K_{E_{\text{ext}}}$	<code>K_EeS</code>	1	1	1
K_C	<code>K_CS</code>	4	4	4
K_O	<code>K_OD</code>	1	1	1
K_D^{diff}	<code>K_D_diff</code>	100	100	100
<i>Translation</i>				
β_M	<code>k_transl_M</code>	0.15	0.15	0.15
β_S	<code>k_transl_S</code>	0.3	0.04 / 0.06	0.24
<i>Internal degradations</i>				
δ_{mRNA}	<code>k_deg_mRNA_M/S</code>	0.23	0.23	0.23
δ_M	<code>k_deg_M</code>	0.005	0.005	0.005
$\delta_{E_{\text{int}}}$	<code>k_deg_E_int</code>	0.001	0.001	0.001
$\delta_{E_{\text{ext}}}$	<code>k_deg_E_ext</code>	0.9	1.3	0.05
δ_C	<code>k_deg_C</code>	0.05	0.05	0.0019
δ_S	<code>k_deg_S</code>	2×10^{-7}	2×10^{-10}	6×10^{-16}
δ_D	<code>k_deg_D</code>	0.02	0.02	0.02
δ_O	<code>k_deg_O</code>	0.02	0.02	0.02
<i>Elimination in the medium</i>				
γ_D	<code>v_max_diff_D</code>	1	1	1
γ_E	<code>v_max_diff_E</code>	0.5	0.5	0.5
γ_C	<code>v_max_diff_C</code>	0.4	0.4	0.4
γ_O	<code>v_max_diff_O</code>	0.01	0.01	0.01

Table 7: Parameters of the genetic circuit for the three experiments: conditioning (Cond.), logic gates (Gates; configurations A and B, identical except β_S : 0.04 for A, 0.06 for B) and pacemaker topology (Pace.).

Parameter	Cond.	Gates	Pace.
<i>Spike (in silico)</i>			
Spike threshold M_{thr}	100	105	97
Output threshold M_{thr}^O	210	170	160
Refractory period	5000 s	5000 s	10000 s
Emission duration	2000 s	2000 s	2000 s
Amplitude D_{diff}	1600	3200	32000
Amplitude $E_{\text{ext,diff}}$	6000	10000	100000
Amplitudes $C/E_{\text{int}}/O$	3000	3000	30000
<i>Diffusion</i>			
Diffusion length λ	1.5×10^{-6} m	1.5×10^{-6} m	1.5×10^{-6} m
Diffusion length λ_C	—	5×10^{-6} m	1.5×10^{-6} m
Spacing	10^{-5} m	10^{-5} m	10^{-5} m
Connection distance d_{max}	3×10^{-5} m	1.2×10^{-5} m	1.2×10^{-5} m
<i>Protocol</i>			
Training / test epochs	40 / 50	150 / 50	150 / 50
Interval between trials	20000 s	20000 s	25000 s
Stimulus duration	2000 s	2000 s	2000 s
Pacemaker (pacemaker_bi)	—	—	8

Table 8: Parameters of the software spike, of the diffusion and of the protocol for the three experiments. The pacemaker (**pacemaker_bi**) exists only on the layered topology, synchronised to the trials.